

Задача: "Магический магазин артефактов: Интеграция и анализ данных"

Описание:

Вы разрабатываете систему для магического магазина, который торгует артефактами из разных эпох и измерений. Данные о товарах хранятся в разнородных форматах: старинные артефакты описаны в XML, современные — в JSON, а легендарные — в зашифрованных текстовых файлах. Ваша задача — создать приложение на C#, которое:

- 1. **Загружает данные** из XML, JSON и текстовых файлов.
- 2. **Объединяет их** в единую коллекцию, используя generics.
- 3. **Выполняет сложные запросы** с помощью LINQ (например, найти артефакты с определенной комбинацией свойств).
- 4. **Экспортирует результаты** в XML/JSON и сохраняет отчеты в текстовые файлы.
- 5. Реализует **интерфейсы** для расширяемости системы.

Детали реализации:

1. Структура данных

- Базовый класс `Artifact` :
 - `Id` (int)
 - `Name` (string)
 - `PowerLevel` (int)
 - `Rarity` (enum: Common, Rare, Epic, Legendary)
 - Метод `Serialize()` (абстрактный).
- Наследники:
 - `AntiqueArtifact` (XML-данные): + `Age` (int), `OriginRealm` (string).
 - `ModernArtifact` (JSON-данные): + `TechLevel` (double), `Manufacturer` (string).
 - `LegendaryArtifact` (текстовые файлы): + `CurseDescription` (string), `IsCursed` (bool).

2. Интерфейсы

- `IExportable` :

```
public interface IExportable {
    string ExportToJson();
    string ExportToXml();
}
```
- `IDataProcessor<T>` :

```
public interface IDataProcessor<T> {
    List<T> LoadData(string filePath);
    void SaveData(List<T> data, string filePath);
}
```

3. Классы для работы с файлами

- `XmlProcessor` (реализует `IDataProcessor<AntiqueArtifact>`):
 - Использует `XmlSerializer` для десериализации XML-файлов.
- `JsonProcessor` (реализует `IDataProcessor<ModernArtifact>`):
 - Использует `Newtonsoft.Json` для работы с JSON.
- `LegendaryProcessor` (для текстовых файлов):
 - Читает строки вида `Name|PowerLevel|Rarity|CurseDescription|IsCursed`, парсит их в `LegendaryArtifact`.

4. Менеджер магазина

Класс `ShopManager` :

- Содержит `List<Artifact>` для объединения всех артефактов.
- Метод `LoadAllData()`, который использует generics для загрузки данных из всех источников.
- Метод `GenerateReport()`, сохраняющий статистику (например, средний `PowerLevel` по редкости) в текстовый файл.

5. LINQ-запросы

Реализовать методы:

- `FindCursedArtifacts()` — все проклятые артефакты с `PowerLevel > 50`.
- `GroupByRarity()` — группировка по редкости с подсчетом количества.
- `TopByPower(int count)` — топ-N артефактов по силе.

Пример входных данных:

XML (antique.xml):

```
<ArrayOfAntiqueArtifact>
  <AntiqueArtifact>
    <Id>...</Id>
    <Name>Amulet of Yendor</Name>
    <PowerLevel>95</PowerLevel>
    <Rarity>Legendary</Rarity>
    <Age>1200</Age>
    <OriginRealm>Arcadia</OriginRealm>
  </AntiqueArtifact>
</ArrayOfAntiqueArtifact>
```

JSON (modern.json):

```
[
  {
    "Id": "...",
    "Name": "Hyper Phase Blaster",
    "PowerLevel": 88,
    "Rarity": "Epic",
    "TechLevel": 9.5,
    "Manufacturer": "TechMage Inc."
  }
]
```

Текстовый файл (legends.txt):

```
Sword of Destiny|100|Legendary|Drains life from the wielder|true
```

Требования:

- 1. Реализуйте все классы и интерфейсы.
- 2. Используйте LINQ для всех запросов.
- 3. Предусмотрите обработку ошибок (например, неверный формат файла).
- 4. Экспортируйте результаты поиска в XML/JSON.
- 5. Создайте консольный интерфейс для взаимодействия с магазином.